Python Script Documentation

Comprehensive report covering student merit filtering, hospital bill calculations, and mobile recharge plan evaluations based on source code executions.

Program 1: Student Eligibility and Merit List

This program processes a list of student dictionaries, filtering candidates who satisfy strict threshold criteria for aggregate marks, subject-wise scores, attendance percentage, and annual family income.

```
MIN_AGGREGATE = 75
MIN_SUBJECT = 60
MIN_ATTENDANCE = 75
MAX_INCOME = 250000

students = [
    {"name": "Anu", "aggregate": 88, "subjects": [85, 90, 88], "attendance": 92, "inco
    {"name": "Ravi", "aggregate": 88, "subjects": [90, 85, 89], "attendance": 92, "inc
    {"name": "Priya", "aggregate": 78, "subjects": [70, 65, 75], "attendance": 80, "in
    {"name": "Kiran", "aggregate": 82, "subjects": [80, 55, 90], "attendance": 95, "in
]

eligible = []
for student in students:
    if (
        student["aggregate"] >= MIN_AGGREGATE
        and all(mark >= MIN_SUBJECT for mark in student["subjects"])
        and student["attendance"] >= MIN_ATTENDANCE
        and student["income"] <= MAX_INCOME
    ):
        eligible.append(student)

eligible.sort(key=lambda s: (-s["aggregate"], -s["attendance"], -s["subjects"][0], s["
```

Program 1 Execution Output

Rank	Student Name	Aggregate	Attendance	Income (Rs.)	Status
1	Ravi	88	92%	150,000	Eligible (Merit Rank 1)
2	Anu	88	92%	180,000	Eligible (Merit Rank 2)
3	Priya	78	80%	200,000	Eligible (Merit Rank 3)
-	Kiran	82	95%	120,000	Ineligible (Failed Subject < 60)

Program 2: Hospital Bill Calculation

Calculates patient concessions based on age and income brackets, computes final balances, and assigns patient classification categories.

```
CONSULTATION_CHARGE = 500
SENIOR_AGE = 60
SENIOR_DISCOUNT = 10
LOW_INCOME_DISCOUNT = 15
MAX_DISCOUNT = 20

def calculate_bill(age, diagnostic, treatment, low_income, payment):
    total = CONSULTATION_CHARGE + diagnostic + treatment
    discount = 0
    if age >= SENIOR_AGE: discount += SENIOR_DISCOUNT
    if low_income: discount += LOW_INCOME_DISCOUNT
    discount = min(discount, MAX_DISCOUNT)
    concession = total * discount / 100
    final_bill = total - concession
    balance = final_bill - payment
    return final_bill, balance, concession
```

Program 2 Execution Output

```
--- HOSPITAL BILL ---
Consultation: Rs. 500
Diagnostics: Rs. 2000
Treatment: Rs. 5000
Total Charges: Rs. 7500
Concession: 20 %
Concession Amount: Rs. 1500.0
Final Bill: Rs. 6000.0
Amount Paid: Rs. 4000
Balance Due: Rs. 2000.0
Patient Classification: Category B - Partially Paid
```

Program 3: Mobile Recharge Plan Evaluator

Evaluates available prepaid recharge plans against customer requirements for validity, budget limits, data, and voice call needs.

```
plans = [
    {"name": "Plan A", "price": 199, "data": 2, "calls": 100, "validity": 28},
    {"name": "Plan B", "price": 299, "data": 5, "calls": 300, "validity": 28},
    {"name": "Plan C", "price": 399, "data": 10, "calls": 500, "validity": 56},
    {"name": "Plan D", "price": 249, "data": 3, "calls": 200, "validity": 28}
]
data_used = 4
calls_used = 250
required_validity = 28
budget = 350
```

Python Program Analysis & Documentation

Technical Report • Generated from Python Source Code & Execution Logs

1. Mobile Plan Comparison & Cost Optimization (`program 3.py`)

This program evaluates multiple telecommunication plans against specific usage parameters (excess data usage and extra call minutes). It computes extra charges based on custom plan rules, aggregates total costs, sorts the plans in ascending order of expenses, and identifies the most economical option.

Core Logic Highlights:

- Iterates over available plans and computes extra data and call charges.
- Dynamically builds an `eligible_plans` dictionary list.
- Sorts options using Python's `lambda` sorting key on total cost.
- Outputs structured comparisons and highlights the optimal choice.

Execution Output Summary:

```
PLAN COMPARISON

Plan: Plan B | Base: Rs. 299 | Extra Chg: Rs. 0 | Total: Rs. 299
Plan: Plan D | Base: Rs. 249 | Extra Chg: Rs. 100 | Total: Rs. 349
Plan: Plan A | Base: Rs. 199 | Extra Chg: Rs. 250 | Total: Rs. 449

MOST ECONOMICAL PLAN: Plan B
Total cost: Rs. 299
```

2. Retail Invoice Generator & Tax Calculator (`program 4.py`)

This script manages a retail checkout workflow. It calculates item totals, checks discount eligibility against predefined purchase thresholds (10% discount if subtotal ≥ Rs. 5,000, capped at Rs. 1,000), applies category-specific tax rates, and formats a clean billing invoice.

Product	Qty	Amount (Rs.)	Discount	Tax	Total (Rs.)
Headphones	2	4000.00	400.00	648.00	4248.00
T-Shirt	2	1600.00	160.00	72.00	1512.00
Rice Bag	1	1000.00	100.00	0.00	900.00

Final Billing Summary:

Subtotal: Rs. 6600.00 • Discount: Rs. 660.00 • Total Tax: Rs. 720.00

Grand Total: Rs. 6660.00

3. Travel Budget Report & Currency Conversion (`program 5.py`)

This program handles multi-currency expense tracking, applying live exchange rates relative to INR, incorporating a standard 2% conversion fee, aggregating category expenses, and performing budget threshold comparisons.

Category	Original Amount	Converted (INR)	Fee (INR)	Total (INR)
Accommodation	500 USD	Rs. 42,500	Rs. 850.0	Rs. 43,350.0
Food	200 EUR	Rs. 18,400	Rs. 368.0	Rs. 18,768.0
Transport	100 GBP	Rs. 11,000	Rs. 220.0	Rs. 11,220.0
Shopping	150 USD	Rs. 12,750	Rs. 255.0	Rs. 13,005.0

Budget Summary & Status:

Allocated Budget: Rs. 100,000 • Total Expenses: Rs. 86,343.0

Status: Within budget (Remaining: Rs. 13,657.0)

Largest Expense Category: Accommodation (Rs. 43,350.0)

Python Program Analysis & Documentation

Technical Report • Comprehensive Script Execution & Output Logs

1. Travel Budget Report (Program 5)

Calculates multi-currency expenses, conversion fees, total costs, and validates against a predefined travel budget.

Source Logic Snippet

```
categories = ["Accommodation", "Food", "T
ransport", "Shopping"] # Processing expen
ses with conversion & fee total_expenses
= 0 # Budget validation and reporting
```

Execution Output Log

```
Category: Accommodation
Original: 500 USD | Converted: Rs. 42500
Fee: Rs. 850.0 | Total: Rs. 43350.0

Category: Food
Original: 200 EUR | Converted: Rs. 18400
Fee: Rs. 368.0 | Total: Rs. 18768.0

Total expenses: Rs. 86343.0
Status: Within budget (Rs. 13657.0
remaining)
```

2. ATM Transaction System (Program 7)

Simulates interactive banking transactions including deposits, withdrawals with limits, balance validations, and summary reporting.

Execution Output & Summary

```
===== ATM TRANSACTION SUMMARY =====
Final Balance: Rs. 3500.0
Total Transactions: 3
- Deposit: Rs. 1500.00
- Withdrawal limit exceeded
- Withdrawal: Rs. 3000.00
```

3. Student Academic Performance Analysis (Program 8)

Processes nested dictionaries containing student subject marks, computes averages, tracks improvement/decline, and categorizes performance.

Student: Ravi

```
Maths - Average: 70.0 (Improvement: 20)
Python - Average: 45.0 (Decline: 10, Weak)
English - Average: 75.0 (Improvement: 10)
Overall Average: 63.33 • Good
```

Student: Sita

```
Maths - Average: 85.0 (Improvement: 10)
Python - Average: 80.0 (Improvement: 10)
English - Average: 90.0 (Improvement: 10)
Overall Average: 85.0 • Excellent
```

4. Shopping Bill & Budget Adjuster (Program 9)

Computes bill totals, applies tier-based discounts (10% for orders ≥ 1000), and dynamically adjusts quantities if budget is exceeded.

Shopping Bill Execution Log

```
Shopping Bill
Book Amount: 300
Pen Amount: 100
Bag Amount: 1000
Total: 1400 | Discount: 140.0
Final Amount: 1260.0 (Budget adjusted successfully)
```